

## گزارش کار تمرین کامپیوتری چهارم درس سیگنال و سیستم ها

امیرحسین صباحی

810101556

### بخش اول

(1.1)

```
function Mapset = make_mapset()
    chars = ['a':'z',' ','.',',','!',';','"',"'"];
    codes = dec2bin(0:31,5) - '0';
    Mapset = cell(2,32);
    Mapset(1,:) = num2cell(chars);
    for i = 1:32
        Mapset{2,i} = codes(i,:);
    end
end
```

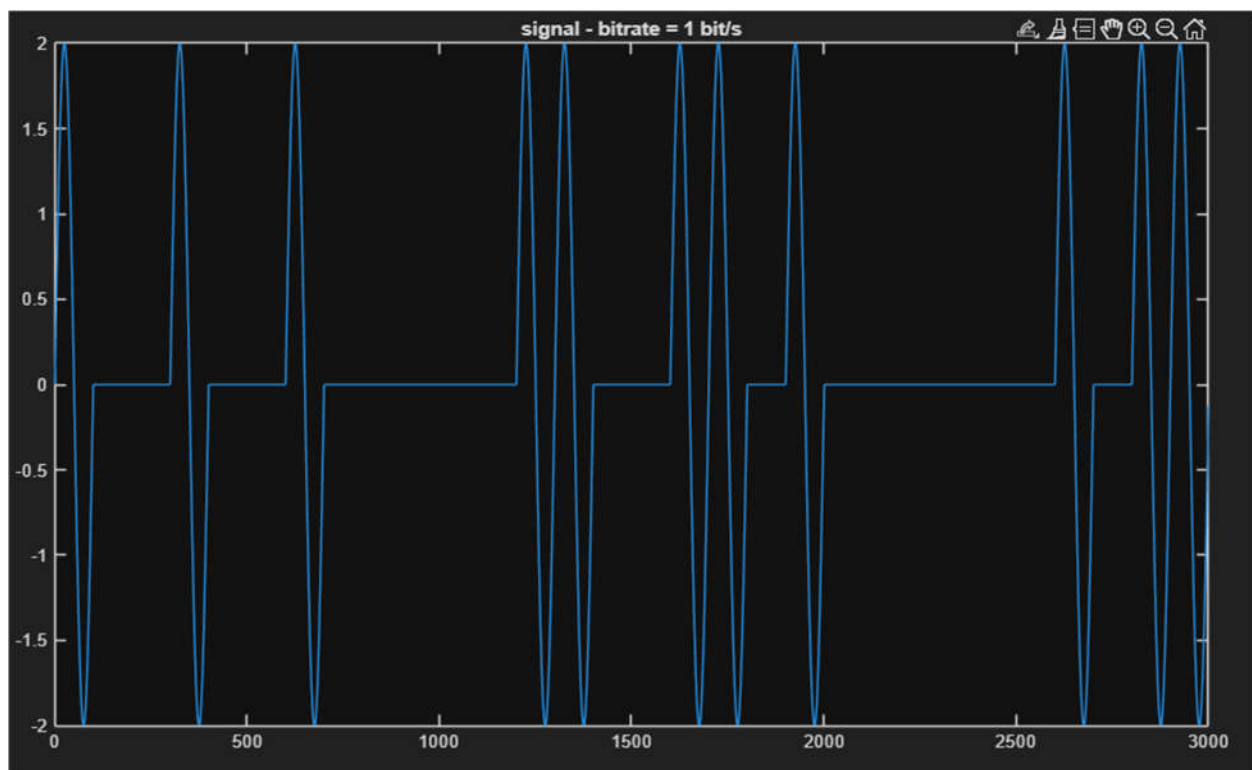
(1.2)

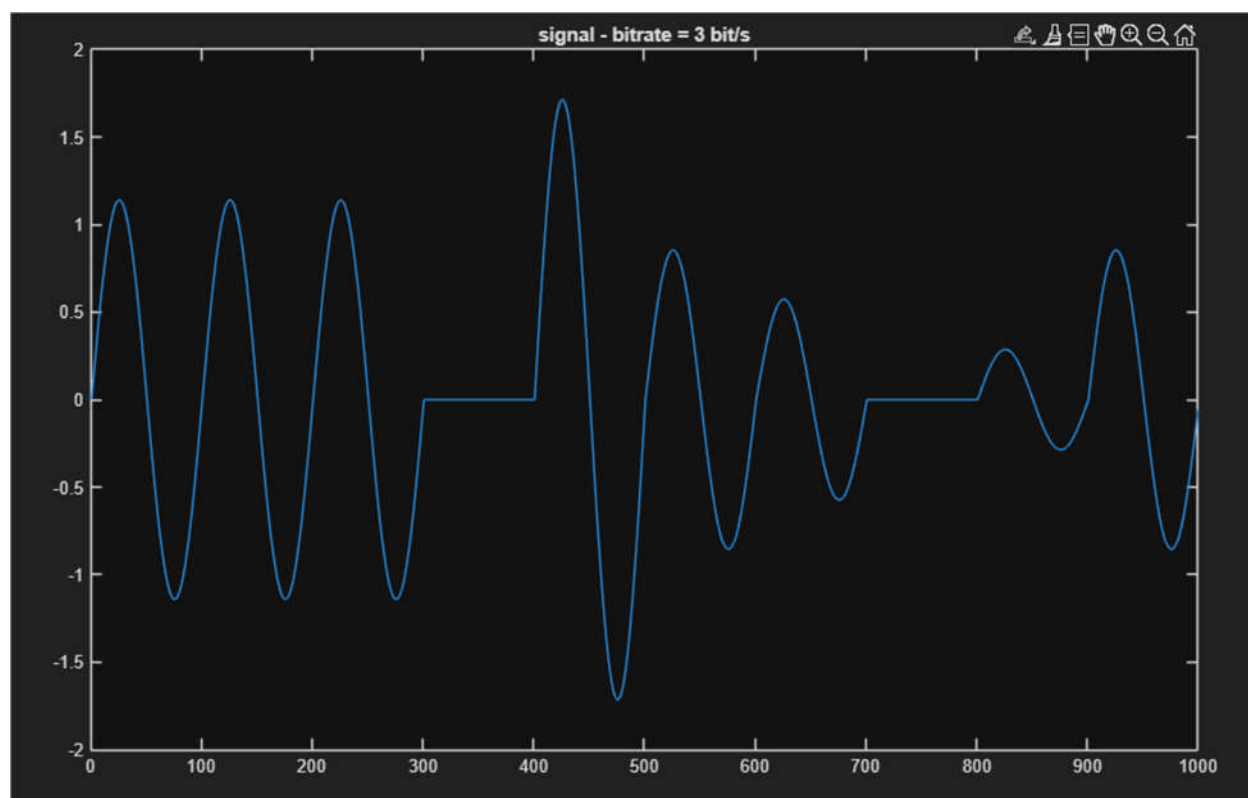
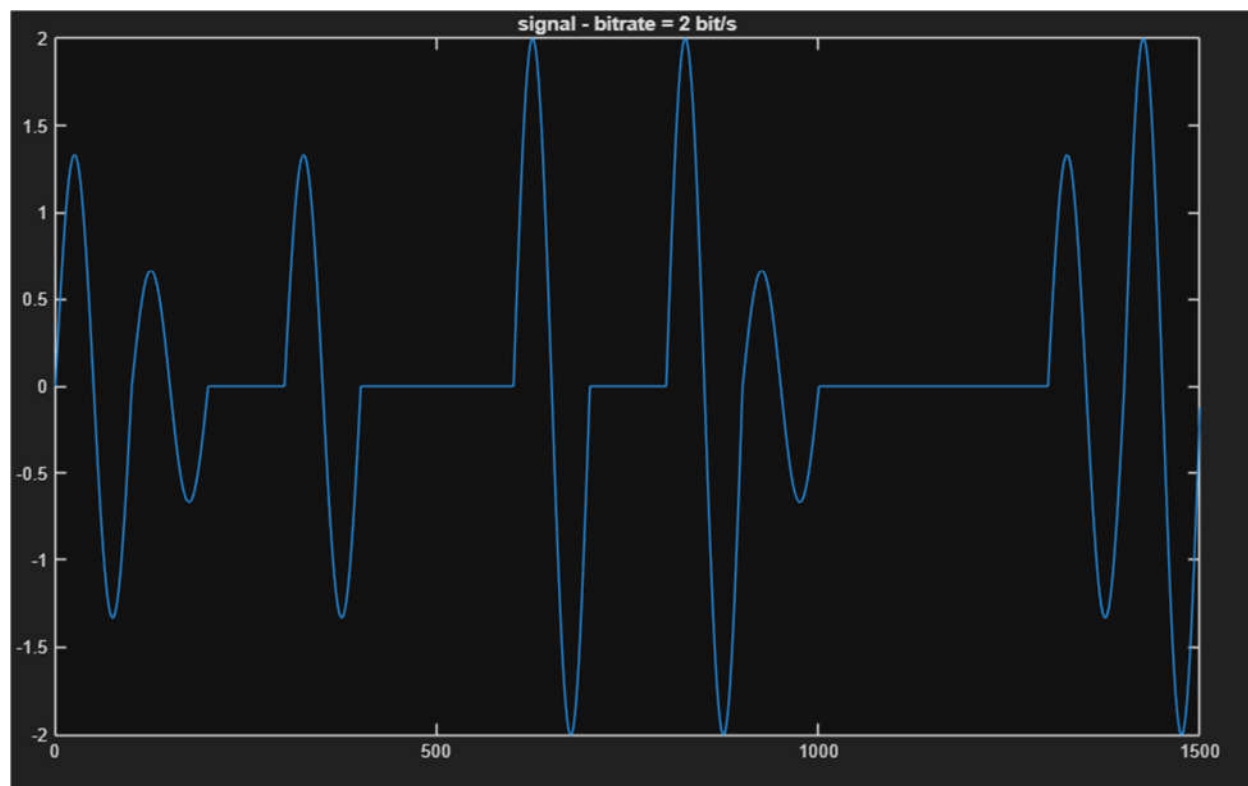
در تابع `coding_amp` ابتدا با فراخوانی `make_mapset` جدول نگاشت کاراکترها به کدهای ۵ بیتی ساخته می‌شود. سپس روی تک تک حروف رشته ورودی `msg` حلقه می‌زنیم، برای هر کاراکتر اندیس مربوطه را در `Mapset` پیدا می‌کنیم و بردار ۵ تایی بیت‌های آن کاراکتر را به انتهای آرایه `bits` اضافه می‌کنیم تا در نهایت کل پیام به صورت یک رشته بیت پشت سر هم درآید. بعد متغیر `R` را برابر `bitrate` قرار می‌دهیم، فرکانس نمونه‌برداری `fs` را ۱۰۰ هرتز می‌گیریم، و با تعریف بردار `t` از ۰ تا تقریباً یک ثانیه، سیگنال پایه `base` را به صورت  $2\sin(2\pi t)$  می‌سازیم که نماینده موج سینوسی یک ثانیه‌ای با دامنه حداکثر ۲ است. سپس آرایه `bits` را طوری `reshape` می‌کنیم که در هر سطر آن `R` بیت کنار هم قرار بگیرند؛ یعنی هر سطر معادل یک «نماد» شامل `R` بیت است و `numSym` تعداد این نمادها خواهد بود. آرایه خروجی `sig` را با طول `numSymN` صفر می‌کنیم تا بعداً قطعه‌های سیگنال را در آن بنویسیم. در ادامه بردار وزن‌های دودویی `w` تشکیل می‌شود (توان‌های ۲ از

$2^{(R-1)}$  تا  $2^0$  و در حلقه‌ای از ۱ تا numSym، مقدار عددی هر نماد با ضرب داخلی سطر bits(n) در w به دست می‌آید (val)، سپس این مقدار بین صفر تا یک نرمال‌سازی می‌شود.  $a = val / (2^R - 1)$ . این a همان دامنه سیگنال در آن بازه یک‌ثانیه‌ای است. در نهایت برای هر نماد، سیگنال یک‌ثانیه‌ای برابر abase ساخته می‌شود و در جای مناسبش داخل sig قرار می‌گیرد، به‌طوری‌که در خروجی، sig یک سیگنال زمانی پیوسته (به صورت نمونه‌برداری شده) است که پیام متنی اولیه را با کدینگ دامنه و بیت‌ریت مشخص شده نمایش می‌دهد.

```
F:\Term7\Signal\coding_amp.m
1 function sig = coding_amp(msg, bitrate)
2     Mapset = make_mapset();
3     bits = [];
4     for k = 1:length(msg)
5         idx = find([Mapset{1,:}] == msg(k), 1);
6         bits = [bits, Mapset{2, idx}];
7     end
8
9     R = bitrate;
10    fs = 100;
11    N = fs;
12    t = (0:N-1)/fs;
13    base = 2*sin(2*pi*t);
14
15    bits = reshape(bits, R, []).';
16    numSym = size(bits, 1);
17    sig = zeros(1, numSym*N);
18
19    w = 2.^(R-1:-1:0).';
20    for n = 1:numSym
21        val = bits(n, :) * w;
22        a = val / (2^R - 1);
23        sig((n-1)*N+1:n*N) = a * base;
24    end
25 end
26
```

```
F:\Term7\Signal\run.m
1      msg = 'signal';
2
3      y1 = coding_amp(msg, 1);
4      y2 = coding_amp(msg, 2);
5      y3 = coding_amp(msg, 3);
6
7      figure;
8      plot(y1);
9      title('signal - bitrate = 1 bit/s');
10
11     figure;
12     plot(y2);
13     title('signal - bitrate = 2 bit/s');
14
15     figure;
16     plot(y3);
17     title('signal - bitrate = 3 bit/s');
18
```





در این بخش، هدف پیاده‌سازی فرایند رمزگشایی (decoding) معکوس تابع `coding_amp` است تا از روی سیگنال زمانی کدگذاری‌شده، دوباره رشته متنی اولیه به‌دست آید. ورودی تابع `decoding_amp` سیگنال زمانی کدگذاری‌شده و نرخ ارسال اطلاعات (`bitrate`) است. ابتدا مانند مرحله کدگذاری، سیگنال پایه به صورت  $2\sin(2\pi t)$  با 100 نمونه در هر ثانیه ساخته می‌شود. سپس طول سیگنال بر 100 تقسیم شده و فرض می‌شود هر 100 نمونه نماینده یک نماد (سمبل) است، بنابراین سیگنال به `numSym` بازه یک‌ثانیه‌ای تقسیم می‌شود. در هر بازه یک‌ثانیه‌ای، حاصل‌ضرب داخلی سیگنال دریافتی با سیگنال پایه محاسبه می‌شود تا مقدار `correlation` به دست آید. برای اینکه نتیجه با حالت پیوسته هم‌مقیاس شود، روی `correlation` ضریب 0.01 اعمال و سپس بر انرژی سیگنال پایه (`corrNorm`) تقسیم می‌شود تا تخمینی از دامنه واقعی نماد (`a_hat`) به‌دست بیاید. این دامنه تخمینی بین صفر و یک محدود می‌شود و سپس با سطوح تئوری دامنه مقایسه شده و نزدیک‌ترین سطح انتخاب می‌شود. شاخص این سطح یک عدد صحیح بین صفر و  $R-1$  بتوان 2 است که نقش همان مقدار دسیمال `R` بیت ارسال‌شده را دارد. این عدد با `dec2bin` به `R` بیت تبدیل می‌شود و بیت‌های هر نماد در یک بردار کلی به نام `bits` در مکان مناسب قرار می‌گیرند. بعد از پایان حلقه روی تمام نمادها، آرایه `bits` که شامل کل بیت‌های پیام است به بلوک‌های ۵ تایی تقسیم می‌شود؛ هر بلوک پنج‌بیتی نماینده یک کاراکتر است. در ادامه، با استفاده از `Mapset` که نگاشت کاراکترها به کدهای پنج‌بیتی را نگه می‌دارد، برای هر پنج بیت، سطر متناظر آن در `Mapset` پیدا شده و کاراکتر مربوطه استخراج می‌شود. با کنار هم قرار دادن این کاراکترها، رشته‌متنی نهایی `msg` ساخته می‌شود که خروجی تابع `decoding_amp` است. برای تست عملکرد تابع، پیام `signal` ابتدا با تابع `coding_amp` برای نرخ‌های ارسال ۱، ۲ و ۳ بیت بر ثانیه کدگذاری شد و سپس سیگنال‌های به‌دست‌آمده به تابع `decoding_amp` داده شدند. در هر سه حالت، خروجی تابع `decoding_amp` دقیقاً برابر رشته `signal` به دست آمد؛ بنابراین می‌توان نتیجه گرفت که الگوریتم رمزگشایی در شرایط بدون نویز به‌درستی عمل می‌کند و فرایند `decoding` به شکل صحیح معکوس کدینگ دامنه پیاده‌سازی شده است.

```

F:\Term7\Signal\decoding_amp.m
1 function msg = decoding_amp(sig, bitrate)
2     Mapset = make_mapset();
3     R = bitrate;
4     fs = 100;
5     N = fs;
6     t = (0:N-1)/fs;
7     base = 2*sin(2*pi*t);
8     numSym = length(sig)/N;
9
10    bits = zeros(1, numSym*R);
11    levels = (0:(2^R-1))/(2^R-1);
12    corrNorm = 0.01*sum(base.^2);
13
14    for i = 1:numSym
15        s = sig((i-1)*N+1:i*N);
16        c = 0.01*sum(s.*base);
17        a_hat = c/corrNorm;
18        a_hat = min(max(a_hat,0),1);
19        [~, idxLevel] = min(abs(a_hat - levels));
20        val = idxLevel - 1;
21        b = dec2bin(val, R) - '0';
22        bits((i-1)*R+1:i*R) = b;
23    end
24
25    bits5 = reshape(bits, 5, []).';
26    codes = zeros(32,5);
27    for k = 1:32
28        codes(k,:) = Mapset{2,k};
29    end
30
31    msgChars = repmat(' ', 1, size(bits5,1));
32    for j = 1:size(bits5,1)
33        idx = find(ismember(codes, bits5(j,:), 'rows'), 1);
34        msgChars(j) = Mapset{1,idx};
35    end
36
37    msg = msgChars;
38 end
39

```

```
%1.4
msg = 'signal';

for R = 1:3
    s = coding_amp(msg, R);
    d = decoding_amp(s, R);
    fprintf('bitrate = %d --> decoded = %s\n', R, d);
end
```

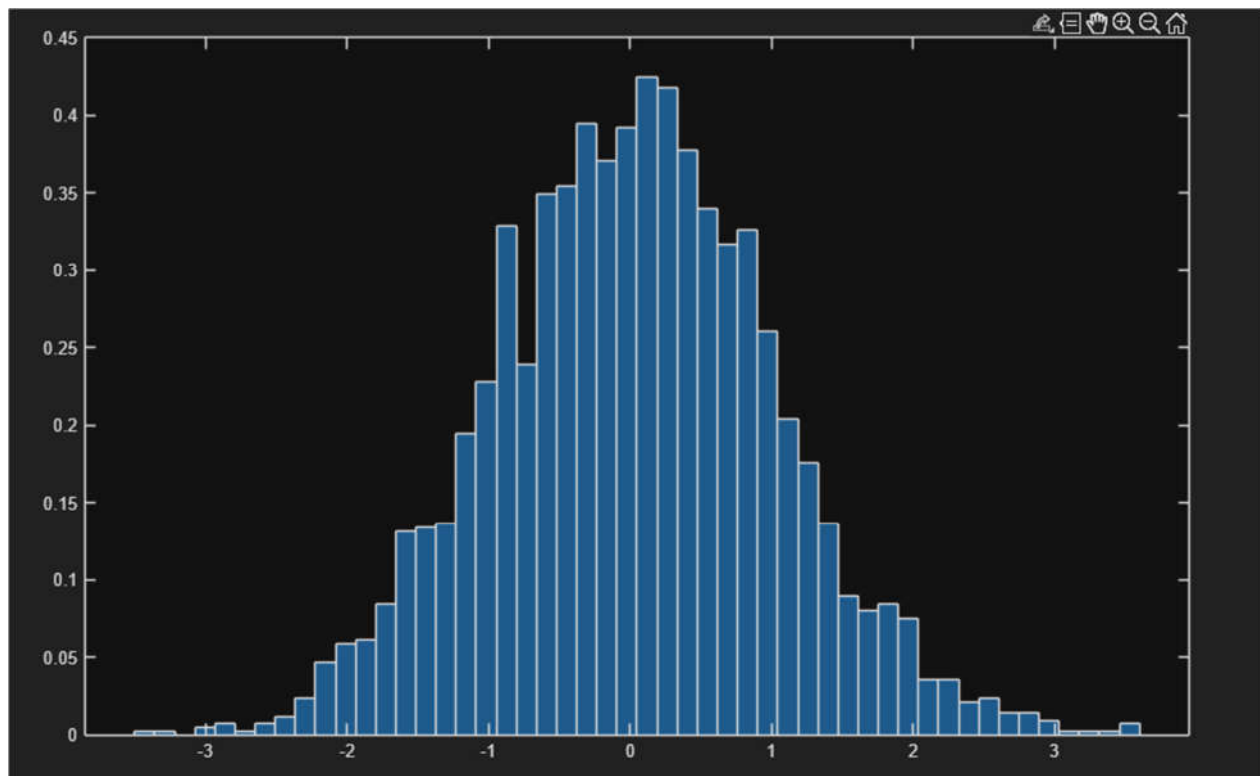
```
Command Window

>> run
bitrate = 1 --> decoded = signal
bitrate = 2 --> decoded = signal
bitrate = 3 --> decoded = signal
>>
```

## 1.5

ابتدا ۳۰۰۰ نمونه نویز با  $\text{randn}(1,3000)$  ایجاد می‌کنیم. سپس با رسم هیستوگرام این نمونه‌ها مشاهده می‌شود که شکل توزیع داده‌ها کاملاً شبیه زنگوله کلاسیک توزیع نرمال است، یعنی بیشترین مقدار داده‌ها در حوالی صفر متمرکز شده‌اند و هرچه از مرکز دورتر می‌شویم فراوانی کاهش می‌یابد؛ این موضوع نشان می‌دهد نویز تولیدشده گوسی است. در ادامه میانگین این نویز با استفاده از  $\text{mean}$  محاسبه می‌شود. مقدار میانگین به دست آمده باید بسیار نزدیک به صفر باشد ( $-0.0145$ )، زیرا توزیع نویزی  $\text{randn}$  ذاتاً میانگین صفر دارد و با افزایش تعداد نمونه‌ها میانگین نمونه‌ای به مقدار تئوری نزدیک‌تر می‌شود. برای بررسی واریانس نیز از دستور  $\text{var}$  استفاده می‌کنیم. مقدار واریانس تقریباً برابر ۱ به دست می‌آید ( $0.9743$ )، که به دلیل محدود بودن تعداد نمونه‌ها کمی از مقدار دقیق یک فاصله دارد ولی همچنان صحت خاصیت واریانس واحد را تأیید می‌کند. این ویژگی‌ها دقیقاً مطابق تعریف نویز گوسی استاندارد هستند.

```
%1.5
x = randn(1,3000);
figure;
histogram(x,50,'Normalization','pdf');
m = mean(x)
v = var(x)
```



```

m =
    -0.0145

v =
    0.9743

```

(1.6)

نویز با استفاده از دستور randn ساخته شد که یک نویز نرمال استاندارد با میانگین صفر و واریانس یک تولید می‌کند. سپس برای تنظیم قدرت نویز، خروجی randn در مقدار  $\sigma$  ضرب شد تا واریانس نویز به  $\sigma^2$  برسد؛ به‌عنوان مثال برای ایجاد نویز با واریانس 0.0001 مقدار  $\sigma$  برابر 0.01 انتخاب شد. نویز تولیدشده به سیگنال اصلی اضافه گردید و در ادامه سیگنال نویزی با تابع decoding\_amp رمزگشایی شد. در هر سه نرخ ارسال اطلاعات یعنی ۱ بیت‌برثانیه، ۲ بیت‌برثانیه و ۳ بیت‌برثانیه، تابع decoding\_amp توانست پیام را بدون

هیچ خطا دوباره به صورت "signal" بازیابی کند. این یعنی با وجود این مقدار بسیار کم نویز، فاصله سطوح دامنه برای هر سه حالت هنوز آنقدر کافی بوده که آستانه‌های تشخیص اشتباه نشوند و سیستم بتواند پیام را به درستی تشخیص دهد.

```
%1.6
msg = 'signal';
sigma = sqrt(0.0001);

for R = 1:3
    s = coding_amp(msg, R);
    noise = sigma * randn(1, length(s));
    rcv = s + noise;
    dec = decoding_amp(rcv, R);
    fprintf('bitrate = %d --> decoded = "%s"\n', R, dec);
end
```

```
bitrate = 1 --> decoded = "signal"
bitrate = 2 --> decoded = "signal"
bitrate = 3 --> decoded = "signal"
>>
```

## 1.7

در این بخش قدرت نویز را در چند مرحله افزایش دادیم؛ به این صورت که بعد از کدگذاری کلمه‌ی «signal»، برای چند مقدار مختلف (مثلاً 0.5، 0.8، 2.5 و 10) نویز گوسی به سیگنال اضافه کردیم و هر بار عملیات decoding را برای سه bit rate برابر 1، 2 و 3 تکرار کردیم. در مقادیر نویز خیلی کوچک، هر سه حالت bit rate 1، 2 و 3 پیام «signal» را بدون خطا بازسازی کردند. [0.4] اما با زیاد شدن تدریجی قدرت نویز، ابتدا برای bit rate 3، [0.8] سپس برای bit rate 2 [2.5] خطا در رمزگشایی ظاهر شد، در حالی که bit rate 1 تا سطوح بالاتری از نویز همچنان پیام را درست استخراج می‌کرد. بنابراین می‌توان نتیجه گرفت که در بین سه حالت، bit rate 1 بیشترین مقاومت را در برابر نویز دارد و با افزایش bitrate سیستم حساس‌تر و نسبت به نویز ضعیف‌تر می‌شود. این نتایج دقیقاً با آنچه در صورت پروژه گفته شده بود هم‌خوانی دارد؛ زیرا با بالا رفتن bit rate تعداد سطوح دامنه بیشتر و فاصله‌ی بین آن‌ها کوچک‌تر می‌شود و همین باعث می‌شود که حتی تغییرات کوچک ناشی از نویز، تصمیم‌گیری گیرنده را دچار خطا کند.

0.8

```
bitrate = 1 --> decoded = "signal"
bitrate = 2 --> decoded = "signal"
bitrate = 3 --> decoded = "sigrc1"
>>
```

2.5

```
bitrate = 1 --> decoded = "signal"
bitrate = 2 --> decoded = "kig!ik"
bitrate = 3 --> decoded = " ipmqc"
>
```

10

```
bitrate = 1 --> decoded = "sigfal"
bitrate = 2 --> decoded = "q,o;ag"
bitrate = 3 --> decoded = ".ohjqc"
>>
```

(1.8

بیشترین واریانس قابل تحمل برای bit rate سه حدود 0.45، برای bit rate دو حدود 1.8 و برای bit rate یک حدود 8.5 به دست آمد.

(1.9

برای مقاوم سازی bit rate در برابر نویز باید توان (Power) فرستنده را افزایش داد تا دامنه سیگنال بزرگ تر شود. وقتی دامنه سیگنال افزایش پیدا می کند، فاصله بین سطوح های مختلف دامنه بیشتر شده و احتمال اشتباه کردن گیرنده در تشخیص سطح صحیح کاهش می یابد.

### 1.10

در کدینگ دامنه، اگر هیچ نویزی در کانال وجود نداشته باشد، از نظر تئوری هیچ محدودیتی برای افزایش بیت‌ریت وجود ندارد. دلیل آن این است که در غیاب نویز، گیرنده می‌تواند هر مقدار دامنه (حتی بسیار نزدیک به هم) را بدون خطا تشخیص دهد، بنابراین می‌توان تعداد سطوح دامنه و در نتیجه نرخ ارسال اطلاعات را به‌طور نامحدود افزایش داد.

### 1.11

عملکرد روش نسبت به نویز مقاوم‌تر نمی‌شود. دلیل آن این است که در فرآیند تشخیص دامنه، شکل موج پایه (Template) فقط برای محاسبه‌ی  $\text{correlation}$  استفاده می‌شود و فرکانس یا تعداد نوسانات سیگنال پایه تأثیری در فاصله‌ی سطوح دامنه و آستانه‌های تصمیم‌گیری ندارد. حساسیت سیستم نسبت به نویز به فاصله‌ی بین سطح‌های دامنه بستگی دارد و چون دامنه‌ی بیشینه همچنان یک است و سطوح دامنه تغییری نمی‌کنند، افزایش فرکانس سیگنال پایه هیچ کمکی به افزایش مقاومت در برابر نویز نمی‌کند. بنابراین روش با ضرب در  $10 \sin(2\pi t)$  به نویز مقاوم‌تر نخواهد شد.

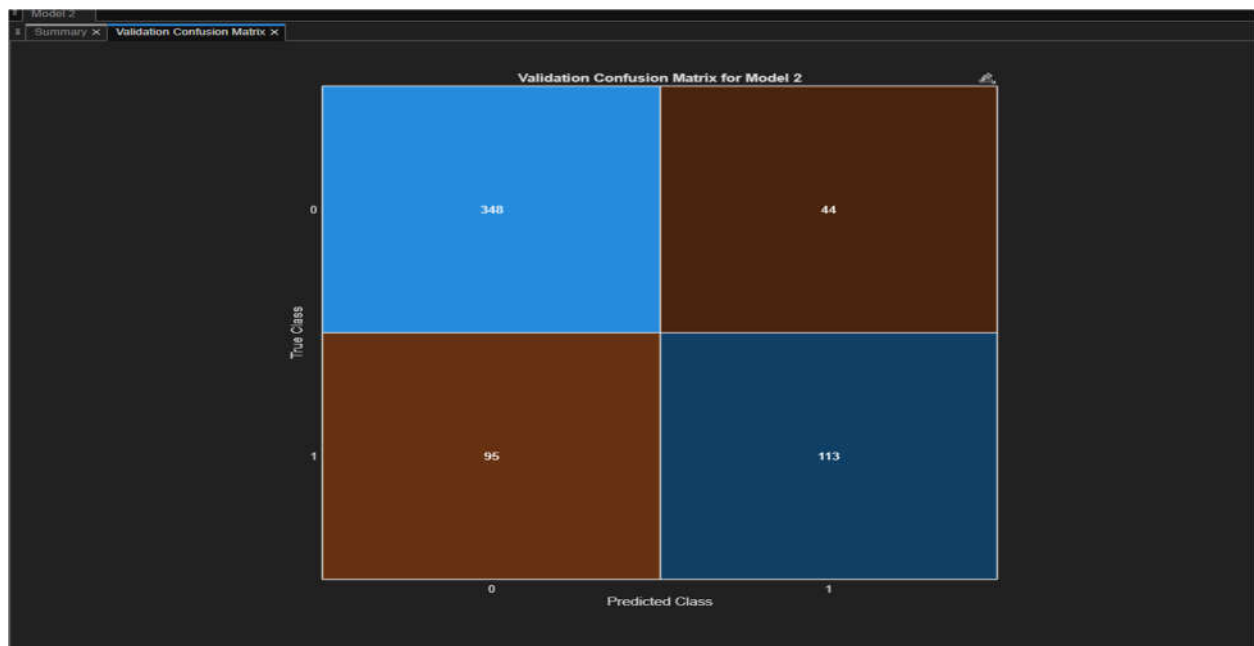
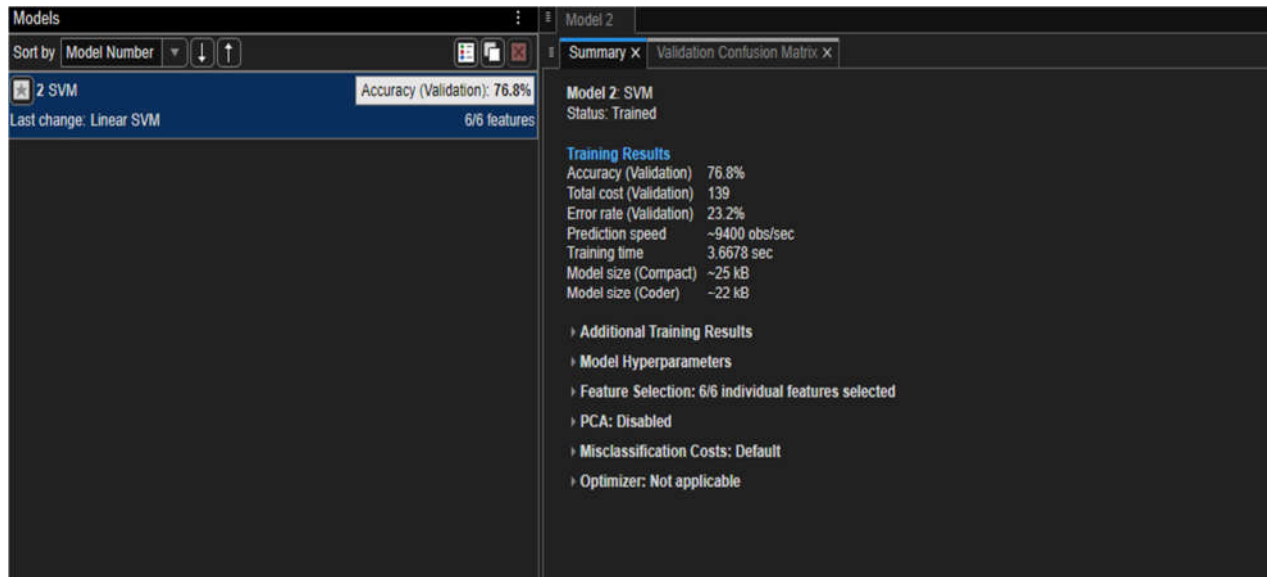
### 1.12

سرعت معمول اینترنت ADSL خانگی معمولاً بین یک تا چند مگابیت بر ثانیه است و حتی در نسل‌های قدیمی‌تر ADSL نیز سرعتی در حدود چند صد کیلوبیت بر ثانیه ارائه می‌شود. در این تمرین، حداکثر نرخ که با آن اطلاعات ارسال کردیم سه بیت بر ثانیه بود، یعنی سرعتی بسیار پایین‌تر از استانداردهای اینترنت واقعی. علت این اختلاف زیاد این است که در این تمرین تنها از کدینگ ساده‌ی دامنه و یک سیگنال سینوسی پایه برای انتقال استفاده کرده‌ایم، در حالی که سیستم‌های واقعی از روش‌های پیچیده‌تر و چندبعدی مانند مدولاسیون‌های چندسطحی، OFDM و کانال‌های فرکانسی استفاده می‌کنند که امکان انتقال همزمان هزاران بیت در ثانیه را فراهم می‌کنند.

## بخش دوم

(2.1)

دقت به دست آمده با مدل Linear SVM در Classification Learner برابر با **76.8%** بود.



3 SVM =Glucose

4 SVM=BloodPressure

5 SVM=SkinThickness

6 SVM=Insulin

7SVM=BMI

8SVM=Age

|                         |                              |
|-------------------------|------------------------------|
| ★ 3 SVM                 | Accuracy (Validation): 74.0% |
| Last change: Linear SVM | 1/6 features                 |
| ★ 4 SVM                 | Accuracy (Validation): 65.3% |
| Last change: Linear SVM | 1/6 features                 |
| ★ 5 SVM                 | Accuracy (Validation): 65.3% |
| Last change: Linear SVM | 1/6 features                 |
| ★ 6 SVM                 | Accuracy (Validation): 65.3% |
| Last change: Linear SVM | 1/6 features                 |
| ★ 7 SVM                 | Accuracy (Validation): 65.7% |
| Last change: Linear SVM | 1/6 features                 |
| ★ 8 SVM                 | Accuracy (Validation): 65.3% |
| Last change: Linear SVM | 1/6 features                 |

**Glucose** ویژگی ای می باشد که وقتی تنها به مدل می دهیم، بیشترین دقت رو می گیریم و بنابراین بیشترین ارتباط رو با دیابتی بودن یک فرد دارد.

## 2.3

در این بخش ابتدا مدل Linear SVM با استفاده از تمام ویژگی‌های موجود در دیتاست diabetes-training آموزش داده شد و سپس از طریق گزینه Export Model با نام TrainedModel در Workspace ذخیره گردید. برای سنجش دقت مدل در مرحله آموزش، داده‌های آموزشی دوباره به مدل داده شد و با استفاده از تابع predictFcn خروجی مدل برای تمام نمونه‌ها پیش‌بینی شد. سپس این پیش‌بینی‌ها با برچسب واقعی ستون آخر جدول مقایسه شد و نسبت نمونه‌های طبقه‌بندی‌شده به‌درستی محاسبه گردید. دقت به دست آمده در این مرحله برابر با **77.5** درصد بود که نشان می‌دهد مدل در مرحله آموزش توانسته است بخش قابل توجهی از نمونه‌ها را به‌درستی شناسایی کند.

```
F:\Term7\Signal\run2.m
1      %2.3
2      X = diabetes_training(:, trainedModel.RequiredVariables);
3
4      yTrue = table2array(diabetes_training(:, end));
5
6      yPred = trainedModel.predictFcn(X);
7
8      acc = mean(yPred == yTrue);
9
10     fprintf('Training accuracy = %.2f%%\n', 100*acc);
11     |
```

```
Command Window
For more information, see how to predict using an exported ...
>> run2
Training accuracy = 77.50%
>> |
```

در این مرحله، داده‌های موجود در فایل diabetes-validation که شامل ویژگی‌های ۱۰۰ فرد جدید به همراه برچسب واقعی دیابتی بودن یا نبودن آن‌ها بود، به مدل آموزش‌دیده قبلی یعنی TrainedModel داده شد. ابتدا فیچرهای مورد نیاز مدل استخراج شد و سپس با استفاده از تابع predictFcn برچسب تمامی نمونه‌های مجموعه اعتبارسنجی پیش‌بینی گردید. پس از مقایسه برچسب‌های پیش‌بینی شده با برچسب‌های واقعی، دقت مدل روی این داده‌های جدید محاسبه شد. نتیجه به دست آمده نشان داد که مدل توانست **78** درصد از نمونه‌های مجموعه validation را به درستی طبقه‌بندی کند. این مقدار به عنوان دقت فاز تست یا دقت ارزیابی گزارش می‌شود.

```
%2.4
```

```
Xval = diabetes_validation(:, trainedModel.RequiredVariables);
yTrue_val = table2array(diabetes_validation(:, end));
yPred_val = trainedModel.predictFcn(Xval);
acc_val = mean(yPred_val == yTrue_val);
fprintf('Validation/Test accuracy = %.2f%%\n', 100*acc_val);
```

#### Command Window

```
Validation/Test accuracy = 78.00%
>>
```